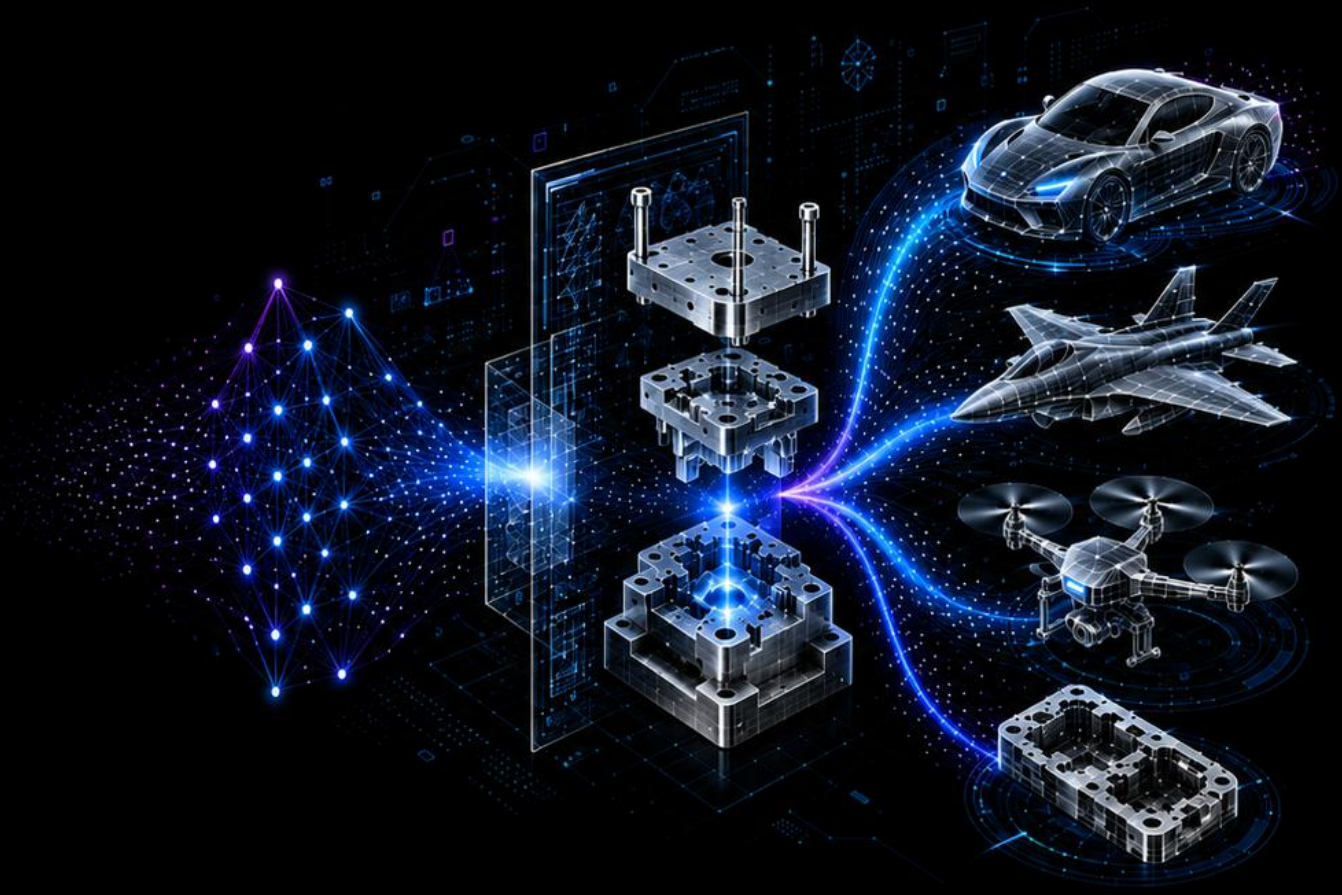
Study

Performance Optimization



하형진

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과

Intelligent Design for Engineering Applications Lab

Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization

- 딥러닝에는 성능을 최적화할 수 있는 다양한 방법이 있어 최적화 방법들을 하나씩 탐구

❖ 데이터를 사용한 성능 최적화

많은 데이터를 수집하는 것, 하지만 데이터 수집이 여의치 않은 상황에서는 임의로 데이터를 생성하는 방법 고려

일반적으로 딥러닝, 머신 러닝 알고리즘은 데이터 양이 많을수록 성능 ↑

따라서, 가능한 많은 데이터(빅데이터)를 수집해야 함

① 데이터 생성 (데이터 수집이 여의치 않을 때)

많은 데이터를 수집할 수 없다면 데이터를 만들어 사용

② 데이터 범위 (Scale) 조정

데이터 값의 크기가 다르면 큰 값이 학습을 지배하여 모델이 불안정해짐

정규화, 표준화, 활성화 함수 등을 통해 과적합을 방지하면

학습이 안정되고 성능이 향상



Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization using Algorithms

- 머신 러닝과 딥러닝을 위한 알고리즘은 상당히 많음, 유사한 용도의 알고리즘들을 선택하여 모델을 훈련시켜보고 최적의 성능을 보이는 알고리즘을 선택

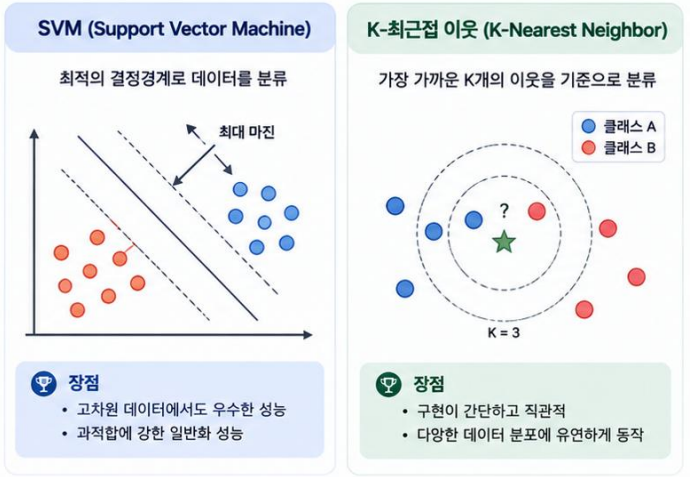
❖ 머신 러닝

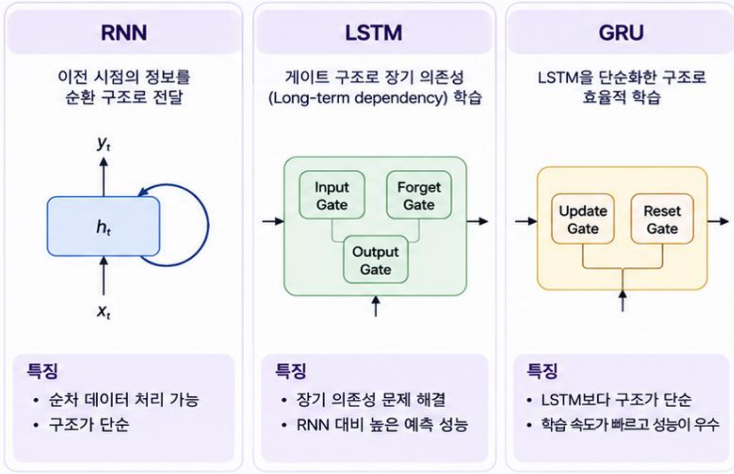
데이터 분류를 위해 SVM, K-최근접 이웃 알고리즘들을 선택하여 훈련

❖ 시계열 데이터

RNN, LSTM, GRU 등의 알고리즘을 훈련시켜 성능이 가장 좋은 모델을 선택

∴ 데이터 특성에 맞는 알고리즘을 선택하여 예측 정확도와 일반화 성능을 효과적으로 향상

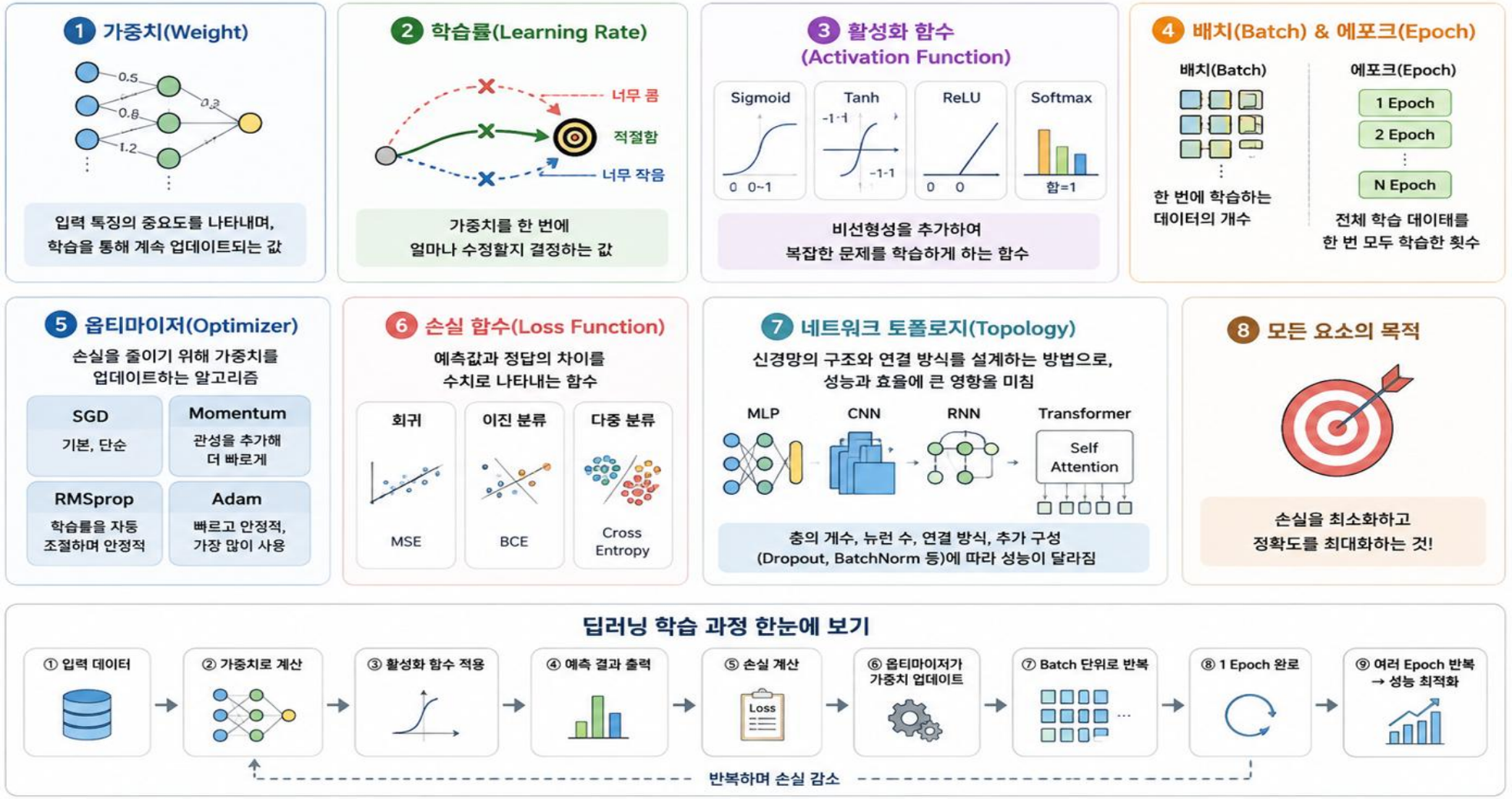




Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization for Algorithm Tuning

- 성능 최적화를 하는 데 가장 많은 시간이 소요되는 부분, 다양한 하이퍼파라미터를 변경하면서 훈련시키고 최적의 성능을 도출



Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization using Ensemble

- 모델을 두 개 이상 섞어서 사용하는 것, 여러 개의 분류기를 생성하고 각 예측들을 결합함으로써 보다 정확한 예측을 도출하는 기법

❖ Voting

서로 다른 알고리즘으로 예측하고 예측한 결과를 가지고 Voting을 통해 최종 예측 결과를 선정하는 방식

❖ Bagging

샘플을 여러 번 뽑아서 각 모델을 학습시킨 다음 결과물을 집계하는 방식

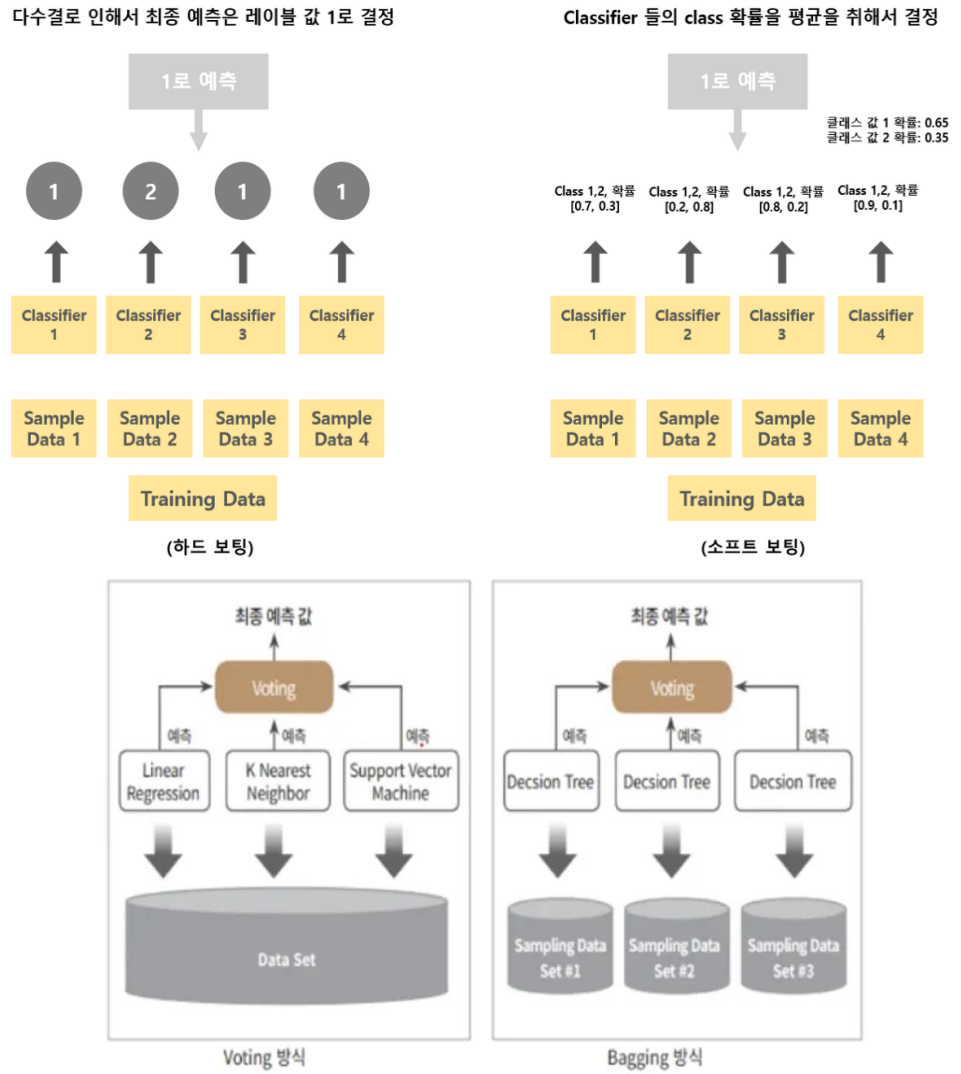
원본 데이터에서 중복을 허용해 무작위로 뽑는 Bootstrap 샘플링을 통해

(조금씩 다른 데이터셋 학습)

e.g. Random Forest

❖ Boosting

Bagging에서 각각의 모델이 독립적으로 학습하는 반면, Boosting은 여러 개의 단순한 모델을 순차적으로 학습시키면서 이전 모델이 틀린 예측에 가중치를 주어 다음 모델이 이를 보완하게 만드는 고성능 예측 방식



Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization using Hardware

- CPU vs GPU, 딥러닝은 대규모 행렬 연산을 반복하기에 GPU를 사용하면 학습 시간(속도)을 크게 단축

❖ CPU

연산을 담당하는 ALU, 명령어를 해석하고 실행하는 Control, 데이터를 담아두는 Cache로 구성. 명령어가 입력되는 순서대로 데이터를 처리하는 직렬 처리 방식

따라서 한 번에 하나의 명령어만 처리하기 때문에 ALU 개수가 많을 필요가 없음

❖ GPU

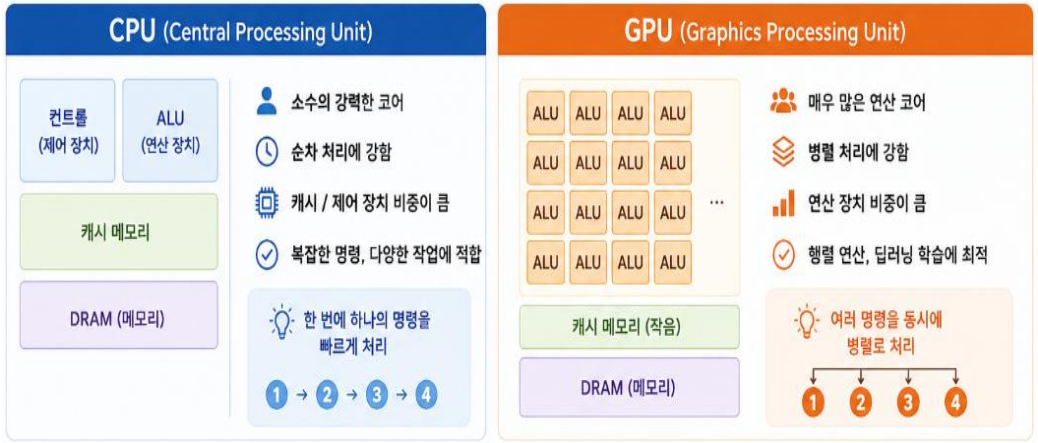
캐시 메모리 비중은 낮고 ALU 개수가 증가, 여러 명령을 동시에 처리하는 병렬 처리 방식에 특화 (CPU로 시간 많이 걸리는 3D 그래픽 작업 등을 빠르게 수행 가능)

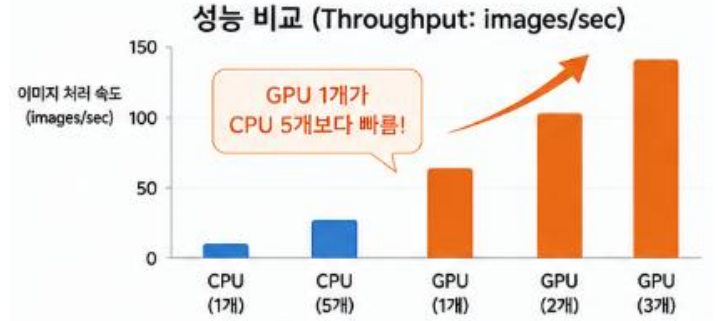
❖ CPU vs GPU

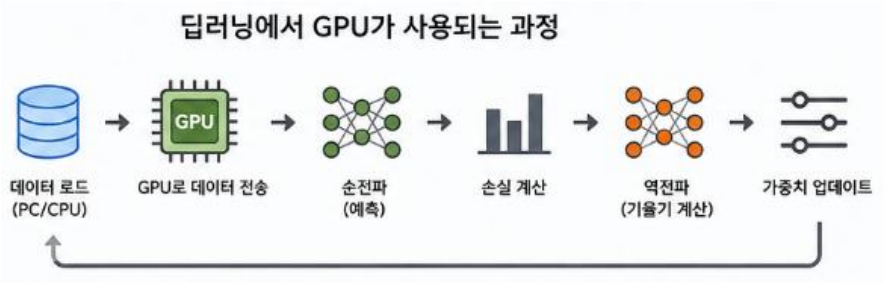
Python, MATLAB처럼 행렬 연산을 많이 사용하는 재귀 연산이 대표적인 '직렬'연산을 수행, 순차적 연산일 때는 CPU가 적합

Backpropagation처럼 복잡한 미적분은 병렬 연산을 해야 속도가 빨라짐

딥러닝은 데이터를 수백~수천만 건까지 다루는데 데이터를 벡터로 변환한 후 연산을 수행한다는 의미, 연산을 수행할 때 CPU에서 한 번에 하나의 명령어만 처리한다면 비효율적이기 때문에 딥러닝에서 GPU 사용은 선택이 아닌 필수







Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization using Batch Normalization

❖ Normalization (정규화)

데이터 값을 일정한 범위 (e.g. 0~1)로 맞추는 과정

특성 간 크기 차이를 줄여 학습 안정화 및 학습 속도 향상

❖ Regularization (규제화)

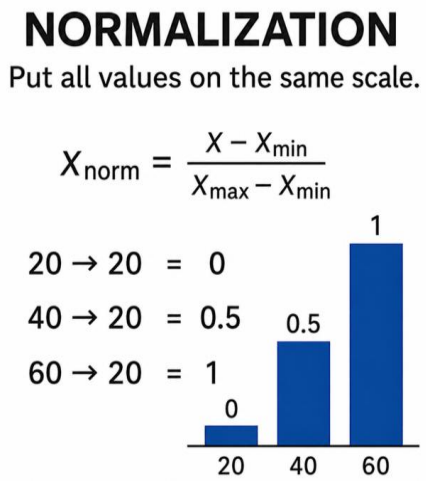
데이터를 평균 0, 표준편차 1이 되도록 변환하는 과정

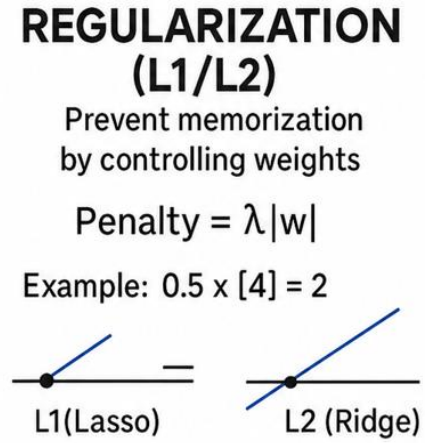
데이터의 분포를 표준화해 모델이 각 특성을 공정하게 학습하도록 도움

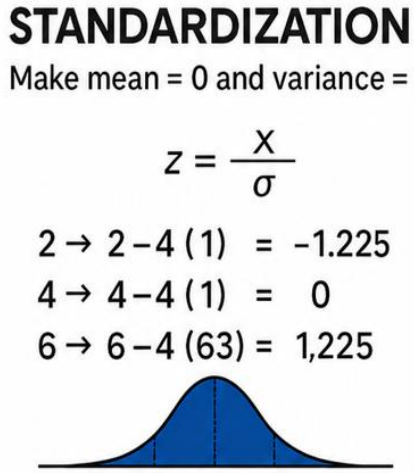
❖ Standardization (표준화)

모델의 복잡도를 제한하기 위해 Penalty를 추가하는 과정

Overfitting을 방지하여 새로운 데이터에서도 일반화 성능 향상







Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization using Batch Normalization

❖ Regularization

① L1 Loss

실제 값과 예측 값 사이의 오차 값의 절대값을 구하고 그 오차들의 합

② L2 Loss

실제 값과 예측 값 사이의 오차 값들의 제곱의 합

③ L1 Loss, L2 Loss 차이

L2 Loss는 Outlier(이상치)에 더 큰 영향을 받음

이상치가 적당히 무시되길 원할 경우 L1 Loss, 이상치에 신경써야 하는 경우 L2 Loss를 사용

④ L1 Regularization (Lasso)

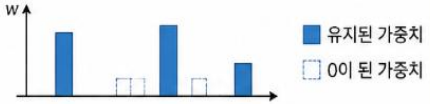
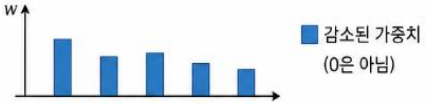
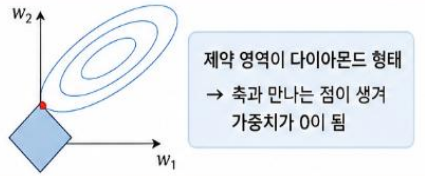
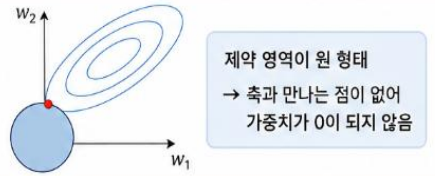
가중치의 절댓값에 Penalty 추가하여 과적합 방지, 불필요한 특징 제거

⑤ L2 Regularization (Ridge)

가중치의 제곱에 Penalty 추가하여 과적합 방지, 모델 안정화

$$L = \sum_{i=1}^{n} |y_i - f(x_i)|$$

$$L = \sum_{i=1}^{n} (y_i - f(x_i))^2$$

항목	L1 Regularization (Lasso)	L2 Regularization (Ridge)
개념	가중치의 절댓값(L1)에 패널티 추가	가중치의 제곱(L2)에 패널티 추가
수식	$L = Loss + \lambda \sum_i \lvert w_i \rvert$	$L = Loss + \lambda \sum_i w_i^2$
가중치 변화	일부 가중치를 0으로 만듦 	모든 가중치를 조금씩 감소 
특징 선택	가능 (Feature Selection) 불필요한 특징을 제거 	불가능 모든 특징을 유지 
장점	모델을 단순화하고 해석이 쉬움	모델을 안정화하고 일반화 성능 향상
기하학적 해석		
대표 이름	Lasso	Ridge

Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization using Batch Normalization

❖ Gradient Vanishing & Gradient Exploding

Gradient Vanishing: 오차 정보를 역전파시키는 과정에서 기울기가 급격히 0에 가까워져 학습이 되지 않는 현상

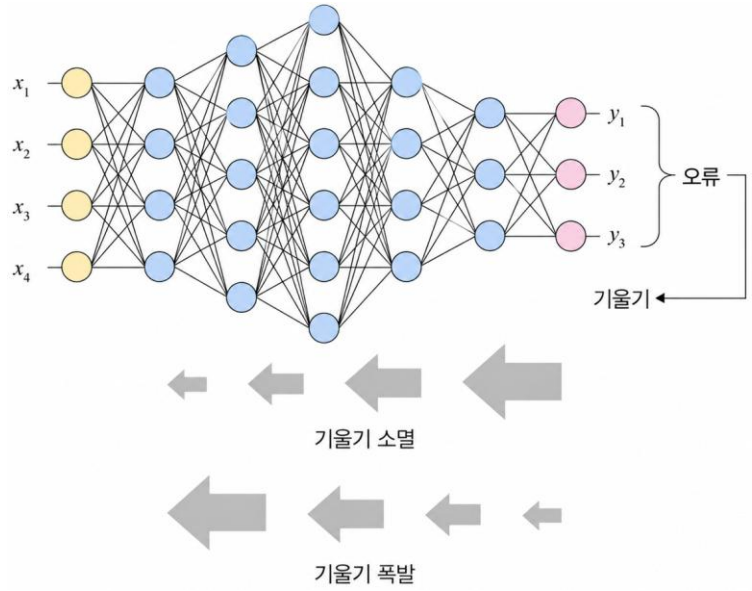
e.g.

[Sigmoid, Tanh 같은 활성화 함수는 미분하면 최대값이 1보다 작은데 이 값들이 계속 곱해지면서 앞쪽 층으로 갈 수록 기울기가 소멸]

Gradient Exploding: 학습 과정에서 기울기가 급격히 커지는 현상

e.g.

[가중치 초기값이 너무 크게 설정되거나 층이 깊은 신경망 및 RNN에서 1보다 큰 기울기 값이 계속 곱해질 때 발생]



Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization using Batch Normalization

- 배치 정규화는 Gradient Vanishing이나 Gradient Exploding 같은 문제를 해결하기 위한 방법

❖ Batch Normalization

기울기 소멸과 폭발 원인은 내부 공변량 변화 때문,

[공변량 변화: 네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌는 현상]

따라서 분산된 분포를 정규분포로 만들기 위해 표준화와 유사한 방식을 Mini-batch에 적용하여 평균은 0으로 표준편차는 1로 유지하도록 함

① 미니 배치 평균 구하기

② 미니 배치의 분산과 표준편차 구하기

③ 정규화를 수행

④ 스케일 조정 (데이터 분포 조정)

$$\mu\beta \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \quad \text{----①}$$

$$\sigma^2\beta \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i-\mu\beta)^2 \quad \text{----②}$$

$$\hat{x}_i \leftarrow \frac{x_i-\mu\beta}{\sqrt{\sigma^2\beta+\epsilon}} \quad \text{----③}$$

$$y_i \leftarrow \gamma\hat{x}_i+\beta \Leftrightarrow BN_{\gamma,\beta}(x_i) \quad \text{----④}$$

입력: $\beta=\{x_1, x_2, \cdots, x_n\}$
학습해야 할 하이퍼파라미터: γ, β
출력: $y_i=BN_{\gamma,\beta}(x_i)$

Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization using Dropout

❖ Overfitting

훈련 데이터셋을 과하게 학습하는 것을 의미, 훈련 데이터셋에 대해 훈련을 계속한다면 오류는 줄어들지만 테스트 데이터셋에 대한 오류는 증가

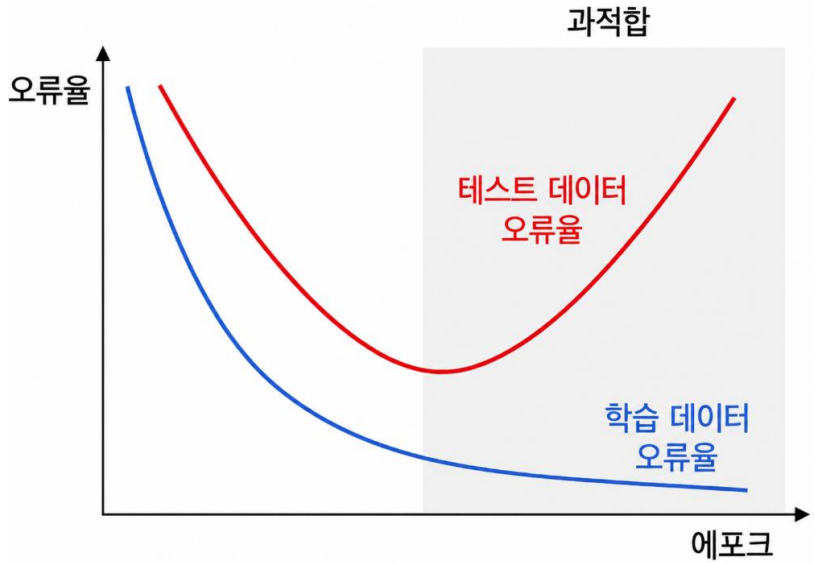
❖ Dropout

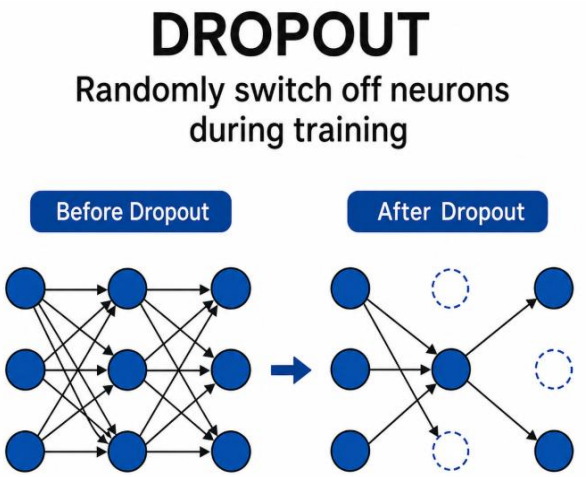
신경망 학습 과정에서 Overfitting을 방지하기 위해 사용되는 기법

학습 단계마다 무작위로 비활성화(=Dropout) 시킴으로써 신경망이 특정 뉴런에 과도하게 의존하는 것을 방지

Dropout에서 어떤 노드를 비활성화할지는 학습할 때마다 무작위로 선정

따라서 다양한 모델을 학습하는 것과 같은 효과를 내는 것이 특징





Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

Performance Optimization using Early Stopping

❖ Early Stopping

모델의 훈련을 조기 중단하는 기법

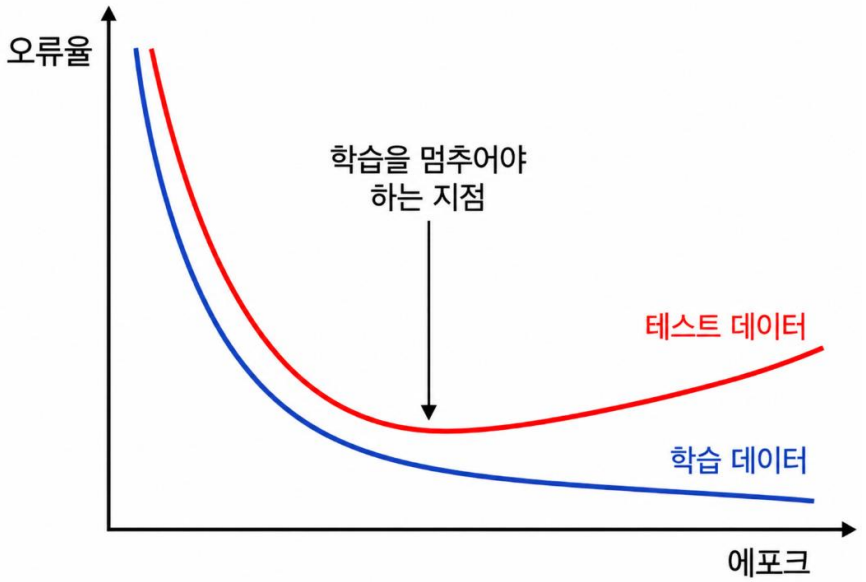
훈련 데이터와 별도로 검증 데이터를 준비하고 매 Epoch마다 검증 데이터에 대한 오차를 측정하여 모델의 종료 시점을 제어

과적합이 발생하기 전까지 학습에 대한 오차와 검증에 대한 오차 모두 감소하지만

과적합이 발생하면 훈련 데이터셋에 대한 오차는 감소하는 반면 검증 데이터셋에 대한 오차는 증가

조기 종료는 검증 데이터셋에 대한 오차가 증가하는 시점에서 학습을 멈추도록 조정

성능이 개선되지 않기 시작하는 순간을 감지하여 성능이 일정 Epoch 동안 개선되지 않으면, 훈련을 조기에 중단하고 최적의 모델을 선택 (Overfitting 가능성 ↓)



Any Questions?

이름 하 형 진

소속 한양대학교 기계공학과

이메일 broha@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab

